UD4_Ejercicios

Ejercicio01. Define un tema e intégralo en tu app:

1. Define un tema con Material 3 Theme Builder, para ello puedes valerte de los recursos ofrecidos para elegir una paleta y tipografía adecuadas.
2. Descarga e integra en tu proyecto los ficheros del tema. (Guarda capturas de los esquemas de la página de Material Design y ponlas en tu proyecto).



3. Modifica tu código para usar los colores del tema.



Color.kt | Type.kt | provider.kt | ViewModel.kt

```kotlin
package com.example.personalprofileapplication4.ui.t

import androidx.compose.ui.graphics.Color

val primaryLight = Color(0xFF226488)
val onPrimaryLight = Color(0xFFFFFFFF)
val primaryContainerLight = Color(0xFFC7E7FF)
val onPrimaryContainerLight = Color(0xFF004C6C)
val secondaryLight = Color(0xFF715C0D)
val onSecondaryLight = Color(0xFFFFFFFF)
val secondaryContainerLight = Color(0xFFFEE186)
val onSecondaryContainerLight = Color(0xFF564500)
val tertiaryLight = Color(0xFF266489)
val onTertiaryLight = Color(0xFFFFFFFF)
val tertiaryContainerLight = Color(0xFFC9E6FF)
val onTertiaryContainerLight = Color(0xFF004B6F)
val errorLight = Color(0xFF895020)
val onErrorLight = Color(0xFFFFFFFF)
val errorContainerLight = Color(0xFFFFDCC4)
val onErrorContainerLight = Color(0xFF6D3A09)
val backgroundLight = Color(0xFFF6FAFE)
val onBackgroundLight = Color(0xFF181C20)
val surfaceLight = Color(0xFFF6FAFE)
val onSurfaceLight = Color(0xFF181C20)
val surfaceVariantLight = Color(0xFFDDE3EA)
val onSurfaceVariantLight = Color(0xFF41484D)
val outlineLight = Color(0xFF71787E)
val outlineVariantLight = Color(0xFFC1C7CE)
val scrimLight = Color(0xFF000000)
val inverseSurfaceLight = Color(0xFF2D3135)
val inverseOnSurfaceLight = Color(0xFFEEF1F6)
```

4. Modifica tu código para usar componentes de material design.

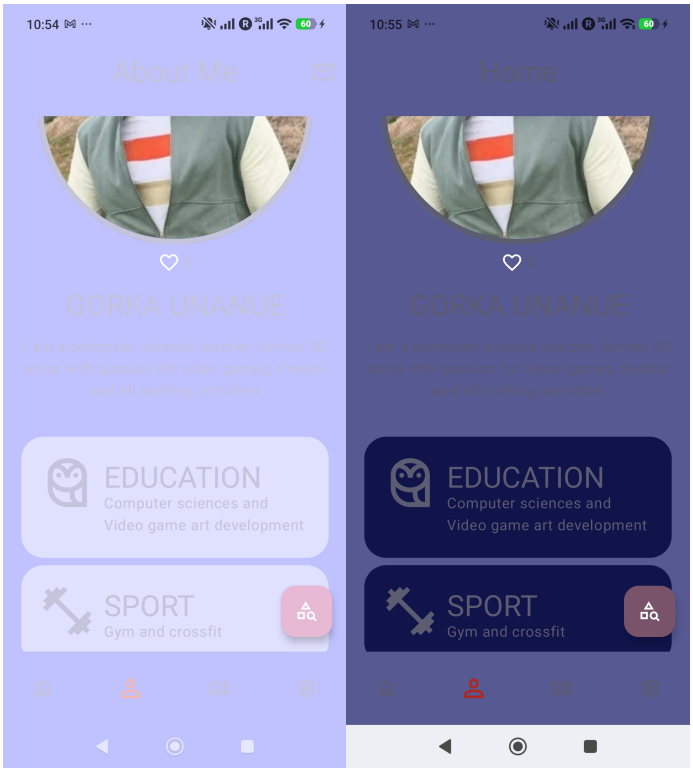
```kotlin
@OptIn( ...markerClass = ExperimentalMaterial3Api::class)
@Composable
fun Topbar(topBarTitle: String, icon: Int?, selectedImage: Int?) {
    val context = LocalContext.current
    CenterAlignedTopAppBar(
        title = { Text(text = topBarTitle, color = MaterialTheme.colorScheme.secondary) },
        actions = {
            if (icon != null) {
                Icon(
                    painter = painterResource(id = icon),
                    contentDescription = null,
                    tint = MaterialTheme.colorScheme.secondary,
```

5. Investiga qué es y aplica Dynamic Color en tu app.

```kotlin
@Composable
fun PersonalProfileApplicationTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }

        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = typography,
        content = content
    )
}
```



Ejercicio02. Define emociones: Elige 3 emociones (por ejemplo: Alegría, tristeza e Ira). Cada emoción representará una pantalla o estado de la app.

- Genera tu paleta: Entra en paletton o huemint y crea una paleta de 7 colores que refleje las emociones elegidas. Ejemplo:

    - Alegría —> Amarillos, naranjas y verdes suaves.
    - Tristeza —> Azules fríos y grises.

- Ira —> Rojos intensos y negros.

Guarda los hexadecimales.

- Crea el tema con Material Theme Builder:

  - Abre Material Theme Builder.
  - Introduce los 3 colores principales de tu paleta (Primary, Secondary, Tertiary).
  - Asigna los demás roles en Advanced —> Color roles.
  - Exporta a Jetpack Compose.

- Integra los ficheros en tu proyecto.

- Añade tu fuente personalizada:

```
val AlegriaFont = FontFamily(Font(R.font.font_alegria))
val TristezaFont = FontFamily(Font(R.font.font_tristeza))
 val IraFont = FontFamily(Font(R.font.font_ira))
val Typography = Typography(
    headlineLarge = TextStyle(fontFamily = AlegriaFont, fontWeight = FontWeight.Bold, fontSize
    = 32.sp),
     bodyLarge = TextStyle(fontFamily = TristezaFont, fontSize = 18.sp),
     labelLarge = TextStyle(fontFamily = IraFont, fontSize = 14.sp) )
```

- Debe contener una Splash Screen animada y tienes que crear una pantalla por emoción. Cada pantalla deberá tener:

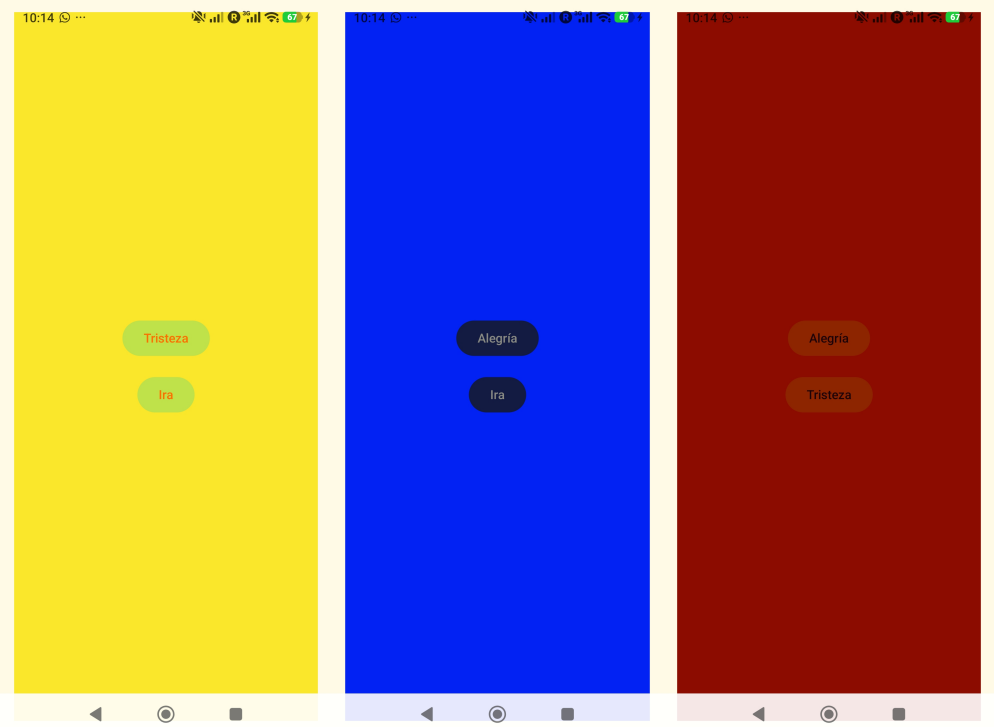
  - Fondo según su color predominante.
  - Tipografía y elementos que reflejen la emoción.
  - Un botón para cambiar a otra emoción.

- Implementa NavHost:

  - Configura rutas: “splash”, “alegria”, “tristeza”, “ira”.
  - Desde el SplashScreen, navega a la primera emoción.

- Implementa modo claro y modo oscuro:

  - Por ejemplo: “alegría brillante” vs “alegría calmada”




```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            EmotionsApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    NavegacionApp()
                }
            }
        }
    }
}


1 Usage
@Preview(showBackground = true)
@Composable
fun NavegacionApp() {
    val navController = rememberNavController()
    val colorBoton: Color
    val colorTexto: Color
    NavHost(
        navController = navController,
        startDestination = "Alegría"
    ) {
        composable( route = "Alegría") {
            Pantalla(
                titulo = "ALEGRÍA",
                color = MaterialTheme.colorScheme.primary,
```

```
            navController = navController,
            destino1 = "Tristeza",
            destino2 = "Ira",
            colorBoton = MaterialTheme.colorScheme.onPrimary,
            colorTexto = MaterialTheme.colorScheme.primaryContainer
        )
    }
    composable( route = "Tristeza") {
        Pantalla(
            titulo = "TRISTEZA",
            color = MaterialTheme.colorScheme.secondary,
            navController = navController,
            destino1 = "Alegría",
            destino2 = "Ira",
            colorBoton = MaterialTheme.colorScheme.onSecondary,
            colorTexto = MaterialTheme.colorScheme.secondaryContainer
        )
    }
    composable( route = "Ira") {
        Pantalla(
            titulo = "IRA",
            color = MaterialTheme.colorScheme.tertiary,
            navController = navController,
            destino1 = "Alegría",
            destino2 = "Tristeza",
            colorBoton = MaterialTheme.colorScheme.onTertiary,
            colorTexto = MaterialTheme.colorScheme.tertiaryContainer
        )
```

```
91          }
92      }
93  }
94
        3 Usages
95  @OptIn( ...markerClass = ExperimentalMaterial3Api::class)
96  @Composable
97  fun Pantalla(
98      titulo: String,
99      color: Color,
100     navController: NavHostController,
101     destino1: String,
102     destino2: String,
103     colorBoton: Color,
104     colorTexto: Color,
105 ) {
106     Column(
107         modifier = Modifier
108             .fillMaxSize()
109             .padding( all = 16.dp)
110             .background(color),
111         horizontalAlignment = Alignment.CenterHorizontally,
112         verticalArrangement = Arrangement.Center
113     ) {
114         Button(
115             onClick = {
116                 navController.navigate( route = destino1)
117             },
```



```
                colors = buttonColors(
                    containerColor = colorBoton,
                    contentColor = colorTexto
                )
            ) {
                Text("$destino1")
            }

            Spacer(modifier = Modifier.height(16.dp))

            Button(
                onClick = {
                    navController.navigate( route = destino2)
                },
                colors = buttonColors(
                    containerColor = colorBoton,
                    contentColor = colorTexto
                )
            ) {
                Text("$destino2")
            }
        }
    }

```